Django-তে `urls.py` হলো সেই ফাইল যেখানে তুমি **URL → View mapping** করো।
অর্থাৎ user browser-এ যে URL লিখে, Django ঠিক করে **কোন view function / class run হবে**।

সহজভাবে:

```
URL  →  View  →  Response
```

Example:

```
http://127.0.0.1:8000/users/
        ↓
users.views.user_list()
        ↓
HTML / JSON Response
```

1️⃣ Basic Django URL Structure

Example `urls.py`

```python
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.home),
    path('about/', views.about),
]
```

Explanation

1️⃣ import path

```python
from django.urls import path
```

`path()` ব্যবহার করে URL define করা হয়।

2️⃣ urlpatterns

```
urlpatterns = [
```

এটা একটি **list** যেখানে সব URL route define করা হয়।

Django server start হলে এই list scan করে।

3️⃣ admin URL

```
path('admin/', admin.site.urls)
```

এটা Django built-in admin panel।

URL:

```
http://127.0.0.1:8000/admin/
```

4️⃣ Home URL

```
path('', views.home)
```

```
http://127.0.0.1:8000/
```

→ `views.home` run হবে।

Example view:

```
def home(request):
    return HttpResponse("Home Page")
```

5️⃣ About URL

```
path('about/', views.about)
```

```
http://127.0.0.1:8000/about/
```

2️⃣ URL With Parameter

Django URL parameter support করে।

Example:

```
path('user/<int:id>/', views.user_detail)
```

URL:

```
/user/5/
```

View:

```
def user_detail(request, id):
    return HttpResponse(f"User ID: {id}")
```

Output:

```
User ID: 5
```

3️⃣ Multiple App URL Structure (Professional)

Big Django project এ URL structure হয় এইভাবে:

```
project
│
├── config
│     urls.py
│
├── users
│     urls.py
│
├── products
│     urls.py
```

Main Project URLs

`config/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('users/', include('users.urls')),
    path('products/', include('products.urls')),
]
```

App URL

`users/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.user_list),
    path('<int:id>/', views.user_detail),
]
```

URL Flow

```
/users/
   ↓
config/urls.py
   ↓
include(users.urls)
   ↓
users/views.py
```

4️⃣ Class Based View URL

Example:

```
from .views import UserListView

urlpatterns = [
```

```
    path('users/', UserListView.as_view()),
]
```

Important:

```
ClassView.as_view()
```

5️⃣ DRF URL Example

DRF এ সাধারণত router ব্যবহার করা হয়।

```
from rest_framework.routers import DefaultRouter
from .views import UserViewSet

router = DefaultRouter()
router.register('users', UserViewSet)

urlpatterns = router.urls
```

API:

```
GET /users/
POST /users/
GET /users/1/
```

6️⃣ URL Naming (Best Practice)

```
path('login/', views.login_view, name='login')
```

Use:

```
reverse('login')
```

or template:

```
{% url 'login' %}
```

7️⃣ Professional Django URL Structure

```
config/
    urls.py

apps/
   users/
       urls.py
   products/
       urls.py
   orders/
       urls.py
```

Main urls:

```
urlpatterns = [
    path('api/users/', include('users.urls')),
    path('api/products/', include('products.urls')),
]
```

🔑 Summary

Component	Work
`path()`	URL define করে
`urlpatterns`	সব URL list
`include()`	app URL connect
`<int:id>`	URL parameter
`name=`	URL reverse

💡 যেহেতু তুমি **DRF + Django production backend** শিখছো, চাইলে আমি তোমাকে দেখাতে পারি:

Professional Django URL Architecture (used by big companies)

```
api/
   v1/
      users/
      auth/
      products/
```

এটা SaaS backend-এ ব্যবহার হয়।
Professional Django project-এ URL architecture সাধারণত **modular, versioned, scalable** হয়। বড় কোম্পানি (SaaS / enterprise backend) এভাবে URL structure বানায় যাতে future-এ API change করা সহজ হয় এবং project বড় হলেও maintain করা যায়।

আমি ধাপে ধাপে explain করছি।

1️⃣ High-Level Professional Django Project Structure

```
project_root
│
├── manage.py
│
├── config/                 # Main Django project
│   ├── __init__.py
│   ├── settings/
│   │    ├── base.py
│   │    ├── dev.py
│   │    └── prod.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── apps/                   # All apps live here
│   ├── users/
│   │    ├── models.py
│   │    ├── views.py
│   │    ├── serializers.py
│   │    ├── urls.py
│   │    └── services.py
│   │
│   ├── auth/
│   │    ├── views.py
│   │    ├── serializers.py
│   │    └── urls.py
│   │
│   ├── products/
│   │    ├── models.py
│   │    ├── views.py
│   │    └── urls.py
│
├── api/
│   ├── v1/
│   │    └── urls.py
│   └── v2/
│        └── urls.py
```

```
│
└── requirements.txt
```

এখানে **3টা important layer আছে**

```
config → api → apps
```

2️⃣ Main URL Entry Point

📁 `config/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.v1.urls')),
]
```

Explanation:

```
http://domain.com/api/
```

সব API request এখানে থেকে শুরু হয়।

3️⃣ API Versioning Layer

📁 `api/v1/urls.py`

```
from django.urls import path, include

urlpatterns = [
    path('auth/', include('apps.auth.urls')),
    path('users/', include('apps.users.urls')),
    path('products/', include('apps.products.urls')),
]
```

এটা **API gateway layer**।

Example URL:

```
/api/v1/auth/login/
/api/v1/users/
/api/v1/products/
```

4️⃣ App Level URLs

📁 `apps/users/urls.py`

```python
from django.urls import path
from .views import UserListView, UserDetailView

urlpatterns = [
    path('', UserListView.as_view()),
    path('<int:id>/', UserDetailView.as_view()),
]
```

Example:

```
GET /api/v1/users/
GET /api/v1/users/10/
```

5️⃣ Final URL Flow

User Request:

```
GET /api/v1/users/5
```

Flow:

```
config/urls.py
      ↓
api/v1/urls.py
      ↓
apps/users/urls.py
      ↓
views.py
      ↓
serializer
      ↓
response
```

6️⃣ Why Big Companies Use This Architecture

1️⃣ API Versioning

```
/api/v1/
/api/v2/
```

Future update করলে পুরানো API break হয় না।

Example:

```
v1 → mobile app
v2 → new frontend
```

2️⃣ Scalability

App wise URL separate।

```
users
orders
payments
notifications
```

3️⃣ Microservice Friendly

Later easily split into services.

Example:

```
auth-service
payment-service
user-service
```

4️⃣ Clean Codebase

সব URL এক জায়গায় না রেখে module wise।

Bad:

```
urls.py = 500 lines
```

Good:

```
small modular urls
```

7️⃣ Example Real Production API URLs

Example SaaS backend:

```
/api/v1/auth/login/
/api/v1/auth/register/

/api/v1/users/
/api/v1/users/10/

/api/v1/products/
/api/v1/products/2/

/api/v1/orders/
/api/v1/orders/5/
```

8️⃣ DRF Router Version (Most Common)

`apps/users/urls.py`

```
from rest_framework.routers import DefaultRouter
from .views import UserViewSet

router = DefaultRouter()
router.register('', UserViewSet)

urlpatterns = router.urls
```

API:

```
GET /api/v1/users/
POST /api/v1/users/
GET /api/v1/users/1/
```

```
PUT /api/v1/users/1/
DELETE /api/v1/users/1/
```

9️⃣ Final Professional URL Architecture

```
config/
   urls.py

api/
   v1/
      urls.py
   v2/
      urls.py

apps/
   users/
       urls.py
   auth/
       urls.py
   products/
       urls.py
   orders/
       urls.py
```

🔑 Key Concepts

Concept	Purpose
`config/urls.py`	Main entry
`api/`	API layer
`v1/`	Versioning
`apps/`	Business modules
`include()`	connect URL

💡 যেহেতু তুমি **Django + DRF + AI surveillance system backend বানাচ্ছো**, চাইলে আমি তোমাকে দেখাতে পারি:

🔥 **Complete Production Django Backend Architecture (used in FAANG-style SaaS)**

```
Django
PostgreSQL
Redis
Celery
JWT
Docker
Nginx
Gunicorn
```

এটা জানলে তুমি **professional backend engineer level architecture** বুঝে যাবে।

হ্যাঁ! 😎 ঠিক ধরেছ তুমি।

Django + DRF professional versioning এ মূল ধারণা হলো:

> **v1 এবং v2 API একই endpoint এর URL থাকতে পারে, কিন্তু ভিন্ন logic / function / view use করতে পারবে।**

অর্থাৎ `/api/users/` v1 এ একরকম behavior দিবে, v2 এ অন্য behavior দিবে, কিন্তু **ক্লায়েন্ট যে URL call করছে তা version দিয়ে নির্ধারিত হয়।**

1️⃣ Example Without Class Versioning

v1 view (`apps/users/views_v1.py`)

```python
from rest_framework.views import APIView
from rest_framework.response import Response

class UserListView(APIView):
    def get(self, request):
        # v1 logic
        users = ["Alice", "Bob"]
        return Response({
            "version": "v1",
            "users": users
        })
```

---

v2 view (`apps/users/views_v2.py`)

```python
from rest_framework.views import APIView
from rest_framework.response import Response

class UserListView(APIView):
    def get(self, request):
        # v2 logic (extra fields, filtering etc.)
        users = [
            {"name": "Alice", "role": "admin"},
            {"name": "Bob", "role": "user"}
        ]
        return Response({
            "version": "v2",
            "users": users
        })
```

2️⃣ URLs

`api/v1/urls.py`

```python
from django.urls import path
from apps.users.views_v1 import UserListView

urlpatterns = [
    path('users/', UserListView.as_view()),
]
```

`api/v2/urls.py`

```python
from django.urls import path
from apps.users.views_v2 import UserListView

urlpatterns = [
    path('users/', UserListView.as_view()),
]
```

3️⃣ Request Behavior

URL	Response
`/api/v1/users/`	`[ "Alice", "Bob" ]`
`/api/v2/users/`	`[{"name":"Alice","role":"admin"}, {"name":"Bob","role":"user"}]`

✅ **v1 এবং v2 একই endpoint, কিন্তু logic সম্পূর্ণ আলাদা।**

4️⃣ Advanced: Same View + Version Check

DRF support করে `request.version`:

```
from rest_framework.views import APIView
from rest_framework.response import Response

class UserListView(APIView):
    def get(self, request):
        if request.version == "v1":
            users = ["Alice", "Bob"]
        elif request.version == "v2":
            users = [
                {"name": "Alice", "role": "admin"},
                {"name": "Bob", "role": "user"}
            ]
        return Response({
            "version": request.version,
            "users": users
        })
```

- 👍 এক view handle করবে multiple versions
- ❌ কিন্তু code messy হলে future maintenance hard হবে
- 💡 Recommended: **v1 এবং v2 আলাদা view রাখা**

✅ Best Practice

1. **v1 → legacy clients**
2. **v2 → new features / optimized response**
3. **Separate urls.py and views.py per version**
4. **Admin, serializers, permissions version wise maintainable**